

第一章 操作系统及 Linux Shell

张兆飞

2026-03-23

第三周 习题

1-17. 在 ___ 处写出执行以下 B shell 命令的结果.

```
$ a=xy
$ b=a_b
$ echo $a $b$a
# __ (1) __
xy a_bxy

$ a=a$b; echo $a $b$a
# __ (2) __
# a=aa_b; b=a_b
aa_b a_baa_b

$ a=$a.b; echo $a $b$a
# __ (3) __
# a=aa_b.b; b=a_b
aa_b.b a_baa_b.b

$ b=$ab; echo $a $b$a
# __ (4) __
# a=aa_b.b; b=
aa_b.b aa_b.b
```

1-18. 在 ___ 处写出执行以下 B shell 命令的结果.

```
$ a=good
$ b=bye
$ echo a1=[$a $b ]
# __ (1) __
a1=[good bye ]

$ echo a2="[$a $b ]"
# __ (2) __
a2="[good bye ]"

$ echo a3='[$a $b ]'
# __ (3) __
a3='[good bye ]'

$ echo a4=\$\a
# __ (4) __
a4=$a
```

3-4. 阅读下列程序后写出应填入 ____ 处的程序语句。

[程序说明] 以下为处理若干个学生成绩的程序, 其功能为

- (1) 从输入文件 student.txt 中读入学生的姓名, 课程 math 和 phys 的成绩, 逐个插入到用结构定义的链表中;
- (2) 从标准输入获得查询任务. 若查询任务的首字符为 n 时, 从链表中统计出学生的人数后 打印; 若查询任务的首字符为 s 时, 再读入学生的姓名, 在链表中查到该学生, 算出总成绩 sum 和平均成绩 average 后打印.

```
#include <stdio.h>

#define S struct student
S
{
    char * name;
    float math, phys;
    S * next;
};
S *First = NULL;

void error(char *format, char *string)
{
    fprintf(stderr, format, string);
    exit(0);
}

void input(char *file)
{
    FILE *fopen(), *fp;
    S *student;
    char name[20];
    float math, phys;

    /* (1) */ fp = fopen(file, "r");
    if(fp == NULL)
        error("cannot open file %s\n", file);

    while(fscanf(fp, "%s%f%f", name, &math, &phys) == 3)
    {
        /* (2) */ student = (S *)malloc(sizeof(S));
        if(student == NULL)
            error("no space for student\n", NULL);

        /* (3) */ student->name = (char *)malloc(strlen(name)+1);
        if(student->name == NULL)
            error("no space for name\n", NULL);

        strcpy(student->name, name);
        student->math = math;
        student->phys = phys;

        /* (4) */ student->next = First; First = student;
    }
}

int getscore(char *name, /* (5) */ float *sum, float *average)
{
    S *student;
```

```

    for(student=First; student!=NULL; student=student->next)
        if(strcmp(name, student->name) == 0)
        {
            /* (6) */ *sum = student->math + student->phys;
            /* (7) */ *average = *sum / 2.0;
            return(0);
        }
    return(-1);
}

int getnum()
{
    int number = 0;
    S *s;
    for( /* (8) */ s=First; s!=NULL; s=s->next)
        number++;
    return(number);
}

main()
{
    char name[20], task[10];
    int number, found;
    float sum, average;

    input("student.txt");
    printf("what task [n|s(number or student)]? ");

    while(scanf("%s", task) > 0)
    {
        switch(task[0])
        {
            case 'n':
                /* (9) */ number = getnum();
                printf("total student number = %d\n", number);
                break;
            case 's':
                printf("who? ");
                scanf("%s", name);
                /* (10) */ found = getscore(name, &sum, &average);
                if(found == 0)
                {
                    printf("student: %s ", name);
                    printf("sum= %8.3f ", sum);
                    printf("average= %8.3f\n", average);
                }
                else
                    printf("not found %s\n", name);
                break;
            default:
                printf("invalid task\n"); exit(0);
        }
        printf("what task [n|s(number or student)]? ");
    }
}

```

3-5. 阅读下列程序, 写出应填入 ____ 处的程序语句.

```
#include <stdio.h>

#define C struct candidate
#define ERROR(S) {printf(S); exit(0);}

C
{
    char name[20];
    int code, vote;
    /* (1) */ C *next;
};

C * Head = NULL;

int getCandidate(FILE * fp)
{
    C * candidate;
    char name[20];
    int nCandidate = 0, code;
    while(fscanf(fp, "%s%d", name, &code) != EOF)
    {
        if(/* (2) */ (candidate = (C *)malloc(sizeof(C))) == NULL)
            ERROR("no space for candidate\n");
        strcpy(candidate->name, name);
        candidate->code = code;
        candidate->vote = 0;
        /* (3) */ candidate->next = Head; Head = candidate;
        nCandidate ++;
    }
    return(/* (4) */ nCandidate);
}

void getVote(FILE * fp, /* (5) */ int *nSenator)
{
    C * candidate;
    int code;
    fscanf(fp, "%d", /* (6) */ nSenator);
    while(fscanf(fp, "%d", &code) != EOF)
    {
        for(/* (7) */ candidate = Head; candidate != NULL; candidate = candidate->next)
            if(code == candidate->code)
            {
                candidate->vote += /* (8) */ 1;
                break;
            }
    }
}

C * electSenator(C * candidate)
{
    int maxVote = 0;
    C * senator;
    senator = NULL;
    for(/* (9) */ candidate = Head; candidate != NULL; candidate = candidate->next)
        if(candidate->vote > maxVote)
        {
```

```

        /* (10) */ senator = candidate;
        maxVote = senator->vote;
    }
    if(/* (11) */ senator != NULL)
        senator->vote = -senator->vote ;
    else
        ERROR("cannot elect senator.\n");
    return(senator);
}

main()
{
    int i, nSenator, nCandidate;
    C * senator;
    FILE *fpVote, *fpCandidate;

    if((fpVote = fopen("vote.txt", "r")) == NULL ||
        !(fpCandidate = fopen("candidate.txt", "r")) )
        ERROR("cannot open files.\n");

    nCandidate = /* (12) */ getCandidate(fpCandidate);
    /* (13) */ getVote(fpVote, &nSenator);

    if(nSenator >= nCandidate)
        ERROR("invalid election\n");

    printf("nS=%d nC=%d\n", nSenator, nCandidate);

    for(i=0; i<nSenator; i++)
        if((senator = electSenator(Head)) != NULL)
            printf("NAME=%-20s CODE=%-5d VOTE=%d\n",
                /* (14) */ senator->name, senator->code, -senator->vote);
}

```